Automatic Differentiation

Jakub Vokoun
Faculty of Applied Sciences
University of West Bohemia
Pilsen, Czech Republic

***Abstract*—This paper provides an explanation of Automatic Differentiation (autodiff) concept for university students at near-bachelor level. Includes comparison with other differentiation methods, very simple autodiff implementation example and more realistic usage in python library.**

***Index Terms*—autodiff, python, computational mathematics**

I. Introduction

Derivative is an indispensable tool in almost any field of mathematics. Its useful property lies in revealing the rate of change of any function. The process of differentiation is very simple, therefore presumably easily encoded into computer program. Since the birth of machine computing various methods were discovered, all with their respective drawbacks. We will explore these methods later. It is necessary to understand the foundational challenges and inner workings of differentiation first.

Derivative may be defined as a limit, such as

$$L = \lim_{h \to 0} \frac{f(a+h) - f(a)}{h} \ , \tag{1}$$

where the limit L exists. In principle, the derivative of any function (if exists) may be computed using this definition. However, since was discovered different approach. When a few simple functions have known derivatives, using them with a set of rules for obtaining derivatives of more complicated functions results in a different, simpler method. The process of finding a derivative is called differentiation.

For example, if we know the derivative of a polynomial

$$(x^a)' = ax^{a-1} \tag{2}$$

where a is an arbitrary integer, and a sum rule

$$(\alpha f + \beta g)' = \alpha f' + \beta g' \tag{3}$$

for any function f, g, and all real numbers α, β, we may obtain derivative for this function:

$$f(x) = 3x^2 + 5x \tag{4}$$

Using both the rules its apparent the result will be:

$$f(x)' = 6x + 5 \tag{5}$$

This simple example demonstrated, that with as little as one known derivative and one rule, we are able to differentiate a whole class of functions, polynomials.

If we add to our set of known derivatives and rules, we are quickly able to differentiate any known function. With large enough set it is only matter of mechanicaly applying rules and known derivatives in the correct order. Something a computer is supposedly very good at, or at least less error-prone than most humans.

The most powerful property of computer is the `if-else` branching logic. Another important part is the ability to perform sequential instructions. This trait might be easily exploitable in order to create a computer program which would take any function as an input and produce its derivative as output.

Most often the function we need to differentiate is in the form

$$f : \mathbb{R}^n \to \mathbb{R}^m \tag{6}$$

where n and m are any positive integers. It might be useful to calculate its partial derivatives or the Jacobian matrix. For some tasks it might be sufficient to only calculate only one partial derivative, a row or column of derivatives.

II. Alternative differentiation methods

Automatic differentiation (autodiff) was not the first method for finding derivatives utilizing computer. In fact it is conceptually more complex than the previous methods.

That is not to say the *older* methods are inherently worse because they were discovered sooner or that are less complex. Every method is useful in particular scenario, however it is important to know about all of them and their advantages and weaknesses.

A. Finite difference method

This method is based on the Eq. (1). The assumption is, that with a small-enough h, the error will be negligible. It is not an analytical, a precise method, rather a numerical tool for obtaining *good-enough* derivative. This method doesn't take advantage of computers properties. It only uses the machine as a mere calculator.

Humans represent numbers most often using arabic numerals. Computers represent and store numbers in different formats. When working with non-integers a floating point arithmetic was introduced, and later standardized [1]. This format has limited precision and for very large or very small (near to zero) numbers is prone to errors.

The great issue arises as the requirement on precision increases. For simplicity we will define our own data type, which has the same drawbacks as `float` but is easier to understand for humans. We will call it `FD2` as *floating decimal 2*, because it is normal arithmetic with precision up to two numbers.

If we tried to calculate derivative for $f(x) = x$, $a = 100$ and $h = 0.01$

$$f(x)' = \frac{f(100 + 0.01) - f(100)}{0.01} \tag{7}$$

we would encounter first error when summing $100 + 0.01 = 100.01$. Due to `FD2` precision, it is trimmed to 100. Next, the function f is identity:

$$\begin{aligned} f(x)' &= \frac{100 - 100}{0.01} \\ f(x)' &= 0 \end{aligned} \tag{8}$$

So, numerically we get the result of derivative to be 0, but we know that derivative of linear function is 1.

B. Symbolic method

A symbolic method was developed to tackle the problem with FD. It is precise. The method requires a set of known derivatives as well as set of rules (chain rule, product rule, …). For a given expression it uses the rules to expand it until only known derivatives are in the expression, then simplify the obtained expression and only then calculate the derivative.

$$\begin{aligned} f(x) &= \frac{\sin(x)}{x} \\ &\text{using quotient rule:} \\ \frac{g(x)}{h(x)} &= \frac{g'(x)h(x) - g(x)h'(x)}{h(x)^2} \\ f'(x) &= \frac{\sin(x)'x - \sin(x)x'}{x^2} \\ &\text{using known derivatives:} \\ \sin(x)' &= \cos(x) \\ x' &= 1 \\ f'(x) &= \frac{\cos(x)x - \sin(x)1}{x^2} \\ &\text{simplify:} \\ f'(x) &= \frac{x\cos(x) - \sin(x)}{x^2} \end{aligned} \tag{9}$$

In Eq. (9) is demonstrated how symbolic method works. This method is used in programs such as Matlab, because it is precise. Note that this method has its drawbacks. The most important one is *expression swell*. The expression might become overly complex during differentiation because operations were not applied in advantageous order. While the result might be simple, the process might be not.

We will demonstrate expression swell on the function $f(x) = (x + 1)^3$. Depending on how the symbolic solver program was written it might first expand the term as seen in Eq. (10). This approach in this particular case is better and leads to faster calculation and less computer memory usage.

$$\begin{aligned} f(x) = (x + 1)^3 &= x^3 + 3x^2 + 3x + 1 \\ f'(x) &= 3x^2 + 6x + 3 \end{aligned} \tag{10}$$

If the solver begins by applying the product rule repeatedly as seen in Eq. (11), expression swell appears rather quickly. Depending on the solver it might simplify during the process so it might be little different than the example. Nevertheless, for more complex expressions it might not even be possible and the problem holds. The example should have demonstrated that even for very simple function, if differentiation done suboptimally, the computational cost is very high.

$$\begin{aligned} f(x) &= (x+1)^3 = (x+1)(x+1)(x+1) \\ f'(x) &= 1(x+1)(x+1) + (x+1)1(x+1) + (x+1)(x+1)1 \\ f'(x) &= (x+1)(x+1) + (x+1)(x+1) + (x+1)(x+1) \\ &\text{using: } (x+1)(x+1) = (x^2 + x + x + 1) \\ f'(x) &= (x^2 + x + x + 1) + (x^2 + x + x + 1) + (x^2 + x + x + 1) \\ f'(x) &= x^2 + x + x + 1 + x^2 + x + x + 1 + x^2 + x + x + 1 \\ f'(x) &= 3x^2 + 6x + 3 \end{aligned} \tag{11}$$

C. Comparison

Both discussed methods have been used and still might be useful to this day. While FD is really simple, can differentiate *"black box"* functions but can lead to wrong results due to computer floating point precision, symbolic method is harder to implement, requires database of known derivatives and rules and is prone to expression swell thus high memory usage but is precise which is highly valuable. The differences are summarized in Table I.

TABLE I
COMPARISON BETWEEN FINITE DIFFERENCES AND SYMBOLIC METHOD.

Property	FD	Symbolic
Accuracy	approximation	exact
Cost	$O(n)$	high
Implementation	easy	moderate
Disadvantages	$h \to 0$:unstable, ε-sensitive	expression swell

III. AUTOMATIC DIFFERENTIATION

Because both finite differences and symbolic method have their respective drawbacks a need for another method arised. The precise year of automatic differentiation discovery is not known, some claims it was as soon as the second half of 1970. Nevertheless, now is it a well-known numerical differentiation method with wide-spread use.

This method have two distinct modes called forward and backward (reverse) mode. Both modes use fundamentally the same idea but in different direction, it will be explained later. However, before exploring the differences between modes let us first understand the core idea.

A. Core principles

Autodiff exploits the fact that every function, no matter how complicated, can be expressed as function composition. This fundamental property is shown in Eq. (12). The other important property is that every composite function may be expressed as its variables and elementary operations[1] that formed them.

[1]Elementary operations are atomic mathematical operations such as addition and multiplication. In the context of autodiff we enlarge this set of functions with well-known derivatives such as cos or log.

$$
\begin{aligned}
f(x) &= x^2 - 4 \\
g(x) &= \sin(x) + x \\
(f \circ g)(x) &= f(g(x)) = \\
f(\sin(x) + x) &= (\sin(x) + x)^2 - 4 \\
(g \circ f)(x) &= g(f(x)) = \\
g(x^2 - 4) &= \sin(x^2 - 4) + x^2 - 4
\end{aligned}
\tag{12}
$$

With the function divided into its compositions, let us use an *evaluation trace*. A special table in which the whole function is recorded. Each row corresponds to an intermediate variable and the elementary operation which created them. These intermediats are typically denoted v_i for functions $f(x) : \mathbb{R}^n \to \mathbb{R}^m$. If we label he variables in $\mathbb{R}^n$ as x_i and the variables in $\mathbb{R}^m$ as y_i, than the intermediate variables indexing follows these rules:

$$
\begin{gathered}
\text{Input variables} \\
v_{i-n} = x_i, \quad i = 1, ..., n \\
\text{Intermediate variables} \\
v_i, \quad i = 1, ..., l \\
\text{Output variables} \\
y_{m-i} = v_{l-i}, \quad i = m-1, ..., 0
\end{gathered}
\tag{13}
$$

Let us demonstrate the evaluation trace on a simple example. We will calculate the function value using evaluation trace for a simple function and their inputs:

$$
\begin{gathered}
y = f(x_1, x_2) = \sin(x_1) - x_1 x_2 + x_2 \\
x_1 = \pi, \quad x_2 = 2 \\
(\text{note } y : \mathbb{R}^2 \to \mathbb{R} \ )
\end{gathered}
\tag{14}
$$

TABLE II
EVALUATION TRACE FOR SIMPLE EXAMPLE FUNCTION.

Variable	Value
$v_{-1} = x_1$	π
$v_0 = x_2$	2
$v_1 = \sin(v_{-1})$	0
$v_2 = v_{-1} v_0$	2π
$v_3 = v_0$	2
$v_4 = v_1 - v_2$	-2π
$v_5 = v_4 + v_3$	$2 - 2\pi$
$y_1 = v_5$	$2 - 2\pi$

We can extend the evaluation trace with *computational graph*. That is a directed acyclic graph (DAG) in which vertices represent variables and edges functions. The graph must be directed and acyclic to ensure the correct flow of computation.

Computational graph is fundamental for our understanding and as a data structure in which can the evaluation trace be algorithmically represented.

Let us build a computational graph for the very same example function from Eq. (14). Note that is strongly corresponds with Table II.

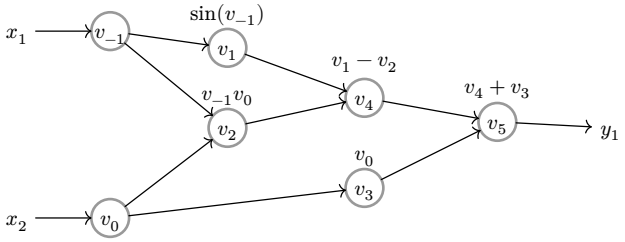

Fig. 1. Computational graph for simple example function.

Decomposition of differentials provided by the chain rule of partial derivatives of composite functions connects the previous concepts and is fundamental for autodiff. One example for function $y(x) = (f \circ g \circ h)$. The variables v_i corresponds with the naming in evaluation trace.

$$
\begin{gathered}
y = f(g(h(x))) = f(g(h(v_0))) = f(g(v_1)) = f(v_2) = v_3 \\
\frac{\partial y}{\partial x} = \frac{\partial y}{\partial v_2} \frac{\partial v_2}{\partial v_1} \frac{\partial v_1}{\partial x} \\
\frac{\partial y}{\partial x} = \frac{\partial f(v_2)}{\partial v_2} \frac{\partial g(v_1)}{\partial v_1} \frac{\partial h(v_0)}{\partial v_0}
\end{gathered}
\tag{15}
$$

To summarize, autodiff makes use of function (de)composition, chain rule and utilizes computational graph. Similarily to symbolic method it requires a set of known derivatives, although it does not need rules as it implicitely works with chain rule.

B. Forward mode: The tangent linear mode

In forward mode the differential decomposition (chain rule) is computed from the inside out (i.e. first is computed the rightmost element $\frac{\partial v_1}{\partial x}$). This mode uses this chain rule recursive relation:

$$
\frac{\partial v_i}{\partial x} = \frac{\partial v_i}{\partial v_{i-1}} \frac{\partial v_{i-1}}{\partial x}, \quad v_n = y
\tag{16}
$$

If this relation is further expanded it gives:

$$
\begin{aligned}
\frac{\partial y}{\partial x} &= \frac{\partial y}{\partial v_{n-1}} \frac{\partial v_{n-1}}{\partial x} \\
&= \frac{\partial y}{\partial v_{n-1}} \left(\frac{\partial v_{n-1}}{\partial v_{n-2}} \frac{\partial v_{n-2}}{\partial x} \right) \\
&= \frac{\partial y}{\partial v_{n-1}} \left(\frac{\partial v_{n-1}}{\partial v_{n-2}} \left(\frac{\partial v_{n-2}}{\partial v_{n-3}} \frac{\partial v_{n-3}}{\partial x} \right) \right) \\
&= ...
\end{aligned}
\tag{17}
$$

Which is exactly the bottom-up (or inside-out) approach. Let us see on an example how it works. We will use the evaluation trace augmented of the value of partial derivative (often called tangent). We use the same function as in Eq. (14) and we are computing the partial derivative $\frac{\partial y}{\partial x_1}$.

$$
\begin{aligned}
y = f(x_1, x_2) &= \sin(x_1) - x_1 x_2 + x_2 \\
x_1 &= \pi, \quad x_2 = 2 \\
\dot{x}_1 = \frac{\partial x_1}{\partial x_1} &= 1, \quad \dot{x}_2 = \frac{\partial x_2}{\partial x_1} = 0
\end{aligned}
\tag{18}
$$

TABLE III
EVALUATION TRACE FOR SIMPLE EXAMPLE PARTIAL DERIVATIVE.

Variable	Value	Tangent	Tangent Value
$v_{-1} = x_1$	π	$\dot{v}_{-1} = \dot{x}_1$	1
$v_0 = x_2$	2	$\dot{v}_0 = \dot{x}_2$	0
$v_1 = \sin(v_{-1})$	0	$\dot{v}_1 = \cos(v_{-1})$	$\cos(\pi) = -1$
$v_2 = v_{-1} v_0$	2π	$\dot{v}_2 = \dot{v}_{-1} v_0 + v_{-1} \dot{v}_0$	$1 \cdot 2 + \pi \cdot 0 = 2$
$v_3 = v_0$	2	$\dot{v}_3 = \dot{v}_0$	0
$v_4 = v_1 - v_2$	-2π	$\dot{v}_4 = \dot{v}_1 - \dot{v}_2$	$-1 - 2 = -3$
$v_5 = v_4 + v_3$	$2 - 2\pi$	$\dot{v}_5 = \dot{v}_4 + \dot{v}_3$	$-3 + 0 = -3$
$y_1 = v_5$	$2 - 2\pi$	$\dot{y}_1 = \dot{v}_5$	-3

This is the essence of forward mode autodiff: At every step calculate the elementary operation as well as the derivatives. Both operations use simple arithmetics or known derivatives therefore the precision is only influenced by computer precision.

In the example it is not evident because the function projects $\mathbb{R}^2 \to \mathbb{R}$, but we did calculate a column in Jacobian matrix. General Jacobian matrix is reminded in Eq. (19).

$$
\mathbb{J} = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \cdots & \frac{\partial y_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial y_m}{\partial x_1} & \cdots & \frac{\partial y_m}{\partial x_n} \end{bmatrix} \tag{19}
$$

Jacobian matrix of our example function is the shape of 1×2. If we fill in the known value we get:

$$
\mathbb{J} = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \frac{\partial y_1}{\partial x_2} \end{bmatrix} = \begin{bmatrix} -3 & \frac{\partial y_1}{\partial x_2} \end{bmatrix} \tag{20}
$$

This beautifully demonstrates the important attribute of the autodiff forward method: It is extremely useful for differentiating functions with few inputs and many outputs. As you know, for function $f : \mathbb{R}^n \to \mathbb{R}^m$ its Jacobian matrix shape is $m \times n$. For $n \ll m$ this mode of autodiff method is the more effective as it requires only n passes.

C. Backward mode: The adjoint mode

With the backward mode, two passes are needed. The first serves to create the evaluation trace and computational graph and is performed as in forward mode. The second pass traverses the graph backwards. This mode uses this chain rule recursive relation:

$$
\frac{\partial y}{\partial v_i} = \frac{\partial y}{\partial v_{i+1}} \frac{\partial v_{i+1}}{\partial v_i}, \quad v_0 = x \tag{21}
$$

Which expanded looks like this:

$$
\begin{aligned}
\frac{\partial y}{\partial x} &= \frac{\partial y}{\partial v_1} \frac{\partial v_1}{\partial x} \\
&= \left(\frac{\partial y}{\partial v_2} \frac{\partial v_2}{\partial v_1} \right) \frac{\partial v_1}{\partial x} \\
&= \left(\left(\frac{\partial y}{\partial v_3} \frac{\partial v_3}{\partial v_2} \right) \frac{\partial v_2}{\partial v_1} \right) \frac{\partial v_1}{\partial x} \\
&= \ldots
\end{aligned}
\tag{22}
$$

And if we define an adjoint for a function $f : \mathbb{R}^n \to \mathbb{R}^m$ where $i = 1, ..., n$ and $j = 1, ..., m$ as:

$$
\bar{v}_i = \frac{\partial y_j}{\partial v_i} \tag{23}
$$

We can rewrite the expanded relation as:

$$
\begin{aligned}
\frac{\partial y}{\partial x} &= \bar{v}_1 \frac{\partial v_1}{\partial x} \\
&= \left(\bar{v}_2 \frac{\partial v_2}{\partial v_1} \right) \frac{\partial v_1}{\partial x} \\
&= \left(\left(\bar{v}_3 \frac{\partial v_3}{\partial v_2} \right) \frac{\partial v_2}{\partial v_1} \right) \frac{\partial v_1}{\partial x} \\
&= \ldots
\end{aligned}
\tag{24}
$$

We will differentiate the same example function. Similarily to forward mode, we must select a variable to derivate by. In forward mode, it is one of income variables, in reverse mode it is one of output variables. In our case, we only have one output variable so it simplifies.

$$
\begin{aligned}
y = f(x_1, x_2) &= \sin(x_1) - x_1 x_2 + x_2 \\
x_1 &= \pi, \quad x_2 = 2 \\
\bar{y}_1 &= 1
\end{aligned}
\tag{25}
$$

TABLE IV
EVALUATION TRACE FOR REVERSE MODE.

↓ Variable	Value	↑ Adjoint	Adjoint Value
$v_{-1} = x_1$	π	$\bar{v}_{-1} = \bar{x}_1 = \bar{v}_1 \cdot \cos(v_{-1}) + \bar{v}_2 \cdot v_0$	$1 \cdot \cos(\pi) + (-1) \cdot 2 = -1 - 2 = -3$
$v_0 = x_2$	2	$\bar{v}_0 = \bar{x}_2 = \bar{v}_3 + \bar{v}_2 \cdot v_{-1}$	$1 + (-1) \cdot \pi = 1 - \pi$
$v_1 = \sin(v_{-1})$	0	$\bar{v}_1 = \bar{v}_4 \cdot 1$	$1 \cdot 1 = 1$
$v_2 = v_{-1} v_0$	2π	$\bar{v}_2 = \bar{v}_4 \cdot -1$	$1 \cdot -1 = -1$
$v_3 = v_0$	2	$\bar{v}_3 = \bar{v}_5 \cdot 1$	$1 \cdot 1 = 1$
$v_4 = v_1 - v_2$	-2π	$\bar{v}_4 = \bar{v}_5 \cdot 1$	$1 \cdot 1 = 1$
$v_5 = v_4 + v_3$	$2 - 2\pi$	$\bar{v}_5 = \bar{y}_1$	1
$y_1 = v_5$	$2 - 2\pi$	$\bar{y}_1$	1

The explanation of this example is necessary. After constructing the forward evaluation trace and computational graph (the two left columns, already calculated in previous examples), we start with the adjoint $\bar{y}_1 = 1$.

We must propagate the adjoint $\bar{y}_1$ to all variables which caused it. In this case, only to v_5. This can be found in the table as $y_1 = v_5$ or in computational graph as all vertices pointing to y_1.

The steps for v_5 are similar, except now the dependencies of it are v_4, v_3. This means we will add to both how much they contribute. For $\bar{v}_4 = \bar{v}_5 \cdot \frac{\partial v_5}{\partial v_4}$ using the recursive formula Eq. (21). The $\bar{v}_5$ is there, because v_4 contributed to v_5. In the partial we are derivating v_5 with respect to v_4. The former comes from the equation $v_5 = v_4 + v_3$ where no additional functions are applied to it. The latter signifies that we are calculating the $\bar{v}_5$ adjoint.

We are essentially deducing how much every variable contributed to the one output variable we set to one. If one variable contributed more than once (in the computational graph have two or more arrows pointing outwards/right) we need to add all its contributions up. That applies to v_0 and v_{-1}. We will skip all the intermediate steps because they are very similar to that already seen.

When calculating $\bar{v}_0$ (which, in fact, is $\bar{x}_2$, thus also $\frac{\partial y_1}{\partial x_2}$), we must add its contribution from $\bar{v}_3$ and $\bar{v}_2$. While $\bar{v}_3$ is trivial, the other variable displays new behaviour. When expanded: $\bar{v}_2 \cdot \frac{\partial (v_{-1} v_0)}{\partial v_0}$ we can see that it is only the result of product rule.

If we were to write this down in jacobian matrix, we would get:

$$\mathbb{J} = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \frac{\partial y_1}{\partial x_2} \end{bmatrix} = [-3,\ 1 - \pi] \tag{26}$$

Where we can see, that in backward mode while we must do one forward pass on the beginning, we than get one row of jacobian matrix on every backward pass.

D. Computational complexity

Now we can compare both modes of autodiff with previously known methods. We will base it on Table I.

TABLE V
COMPARISON BETWEEN METHODS.

Property	FD	Sym	AD (FM)	AD (BM)
Accuracy	approximation	exact	exact	exact
Cost	$O(n)$	high	moderate	moderate
Implementation	easy	moderate	moderate	moderate
Disadvantages	unstable	expression swell	memory overload	control-flow complexity

The main drawback for autodiff (especially backward mode) is the high memory usage. It needs to hold the computational graph which could be rather large.

IV. Minimal implementation

Following sections demonstrate how could autodiff be naively implemented inside `python`. In implementation of both modes, the same example function is used as previously.

A. Forward mode

```python
import math

class Variable:

    def __init__(self, value, tangent):
        self.value = value
        self.tangent = tangent

    def __add__(self, other):
        value = self.value + other.value
        tangent = self.tangent + other.tangent
        return Variable(value, tangent)

    def __sub__(self, other):
        value = self.value - other.value
        tangent = self.tangent - other.tangent
        return Variable(value, tangent)

    def __mul__(self, other):
        value = self.value * other.value
        tangent = self.tangent * other.value + \
                  other.tangent * self.value
        return Variable(value, tangent)

    def __repr__(self):
        return f"(value: {self.value}," \
               f" tangent: {self.tangent})"

# only works if x is of type Variable
def sin(x):
    return Variable(
            math.sin(x.value),
            math.cos(x.value))

x1 = Variable(math.pi, 1.0)
x2 = Variable(2.0, 0.0)
y = sin(x1) - x1 * x2 + x2

print(f"{x1 = },\n{x2 = }")
print(f"{y = }")
```

Listing 1. Forward mode autodiff naive implementation in python.

In the code above can be seen the principle of forward mode: The function is evaluated and the partial is computed along at the same time.

The program creates a new data type (`Variable`) and via functions determine how it responds to standard operators $(+, -, \cdot)$. The function for sinus is created in the same way. The function `__init__` explains python how to initialize the variable, `__repr__` tells it how to represent it in text.

The output of this program is:

```
x1 = (value: 3.141592653589793, tangent: 1.0),
x2 = (value: 2.0, tangent: 0.0)
y = (value: -4.283185307179586, tangent: -3.0)
```

Listing 2. The output of FM autodiff example.

Which gives to show that these results are the same as ours previously ($2 - 2\pi \approx -4.28$). Note that we decided that the partial would be calculated with respect to x_1 simply by setting its tangent to 1.

B. *Backward mode*

```
import math

class Variable:

    def __init__(self, value, adjoint=0.0):
        self.value = value
        self.adjoint = adjoint

    def backward(self, adjoint):
        self.adjoint += adjoint

    def __add__(self, other):
        variable = \
          Variable(self.value + other.value)

        def backward(adjoint):
            variable.adjoint += adjoint
            self_adjoint = adjoint * 1.0
            other_adjoint = adjoint * 1.0
            #
            self.backward(self_adjoint)
            other.backward(other_adjoint)

        variable.backward = backward
        return variable

    def __sub__(self, other):
        variable = \
          Variable(self.value - other.value)

        def backward(adjoint):
            variable.adjoint += adjoint
            self_adjoint = adjoint * 1.0
            other_adjoint = adjoint * -1.0
            #
            self.backward(self_adjoint)
            other.backward(other_adjoint)

        variable.backward = backward
        return variable

    def __mul__(self, other):
        variable = \
          Variable(self.value * other.value)

        def backward(adjoint):
            variable.adjoint += adjoint
            self_adjoint = adjoint * other.value
            other_adjoint = adjoint * self.value
            #
            self.backward(self_adjoint)
            other.backward(other_adjoint)

        variable.backward = backward
        return variable


    def __repr__(self) -> str:
        return f"(value: {self.value}," \
               f" adjoint: {self.adjoint})"


def sin(x):
    variable = Variable(math.sin(x.value))

    def backward(adjoint):
        variable.adjoint += adjoint
        x.backward(adjoint * math.cos(x.value))

    variable.backward = backward
    return variable

x1 = Variable(math.pi)
x2 = Variable(2.0)

y = sin(x1) - x1 * x2 + x2
y.backward(1.0)

print(f"{x1 = },\n{x2 = }")
print(f"{y = }")
```

The backward mode requires more complex implementation. We created a data type of `Variable` again, but this time it has function `backward()`, which is used to accumulate all contributions. However, this function is overwritten when mathematical operation is applied to the variable.

For example, when the multiplication takes place, it creates new variable to hold the value (forward pass) as well to hold the backward function. And the backward function is nothing special, it just specifies how adjoint is calculated. It uses the same principle as shown in Table IV.

```
x1 = (value: 3.141592653589793, adjoint: -3.0),
x2 = (value: 2.0, adjoint: -2.141592653589793)
y = (value: -4.283185307179586, adjoint: 1.0)
```

Listing 3. The output of BM autodiff example.

Please note, that it calculated the jacobian row $[-3,\ 1-\pi]$ as adjoints of x_1, x_2. The other interesting thing is that in order to compute that, we needed to call `backawrd()` method on the `y` variable. That call started a recursive process which filled adjoints of all variables.

V. Using existing autodif libraries in python

More realistic use of autodiff is via library. In this example we are using PyTorch for both forward and backward mode.

```
import torch
from torch.func import jacfwd

# function: y = sin(x1) - x1*x2 + x2
def f(x):
    x1, x2 = x
    return torch.sin(x1) - x1 * x2 + x2

# input
x = torch.tensor([torch.pi, 2.0],
requires_grad=True)

# FM
jac_fwd = jacfwd(f)(x)

# BM
y = f(x)
y.backward()
grad_rev = x.grad

print("FM:")
print(jac_fwd)

print("\nBM:")
print(grad_rev)py
```

Listing 4. Autodiff using pytorch.

We defined a function `f(x)` because pytorch requires function with only one variable input. That is why we *unpack* it into `x1, x2` and use in the same function as previously. Then we prepare these under `# input`.

For forward mode pytorch provides a function `jacfwd` (Jacobian using forward mode) which needs the function and the input variables. Similar to our implementation, but it does two passes to calculate both columns in the jacobian.

For the backward mode we define the output variable `y`, call backward on it and we are interested in adjoints in `x`, which is stored as `x.grad`. When we print these, we get:

```
FM:
tensor([-3.0000, -2.1416],
grad_fn=<ViewBackward0>)

BM:
tensor([-3.0000, -2.1416])
```

Listing 5. The output of PyTorch autodiff.

VI. Conclusion

Automatic differentiation is a powerful alternative to formerly widely used finite differences and symbolic method. Its strength lies in avoiding approximation errors and expression swell, both drawbacks of mentioned methods. The core principle is systematic application of the chain rule on elementary operations.

With mathematical foundation and simple implementation of both modes, this paper demonstrated how the method works. Forward method is intuitive, effective for functions with few inputs. Backward mode is well-suited for scalar-output problems, common in optimalization and machine learning.

Overall, automatic differentiation is conceptually simple, computationally efficient and therefore essential for scientific computing.

VII. Further reading

For those interested in autodiff, I can recommend these sources: [2], [3], [4]

This seminar work with all its source code and working python examples is on github.com/vokymir/kma-nm/tree/master/autodif.

References

[1] I. of Electrical and E. Engineers, "IEEE Standard for Floating-Point Arithmetic." [Online]. Available: https://standards.ieee.org/ieee/754/6210/

[2] Wikipedia contributors, "Automatic differentiation — Wikipedia, The Free Encyclopedia." [Online]. Available: https://en.wikipedia.org/w/index.php?title=Automatic_differentiation&oldid=1339245165

[3] Andrew Holmes, "What's Automatic Differentiation?." [Online]. Available: https://huggingface.co/blog/andmholm/what-is-automatic-differentiation

[4] Carnegie Mellon University, "Deep Learning Systems - Algorithms and Implementation." [Online]. Available: https://dlsyscourse.org/